XlsxToJson 적용 가이드

XlsxToJson은 기획 데이터 엑셀(GameData.xlsx) 하나에서 시트별 JSON, 그 JSON을 읽는 C++/C# 행 코드, enum·상수 정의를 한 번에 만드는 변환 도구입니다. 엑셀 헤더가 스키마이고 코드는 그 스키마에서 나오므로 데이터와 코드가 항상 일치합니다.

1. 전체 구조

```
GameData.xlsx ──▶ XlsxToJson ──┬──▶ <시트>.json                                (--out)
                               ├──▶ <시트>.gen.h,
Enums/EnumStrings/Consts.gen.h, GeneratedCommon.h   (--cpp-out)
                               └──▶ <시트>.gen.cs, Enums.gen.cs, Consts.gen.cs
(--cs-out)
```

- 시트 하나가 JSON 파일 하나, C++ 구조체 하나(`S<시트>Row`), C# 클래스 둘(`<시트>Row` , `<시트>Table`)이 됩니다.
- `Enum` , `Const` 시트는 JSON이 아니라 형식 정의입니다. enum과 상수 코드만 만듭니다.
- 코드 생성은 출력 폴더를 지정한 언어만 합니다. `--cpp-out` 만 주면 C++, `--cs-out` 만 주면 C#, 둘 다 주면 둘 다 만듭니다.
- 오류가 하나라도 있으면 모든 오류를 출력하고 아무 파일도 쓰지 않습니다.

2. 프로젝트 구성

파일	역할
`XlsxToJson.cpp`	진입점. 명령행 해석, 시트 읽기 순서, 파일 쓰기(원자적 쓰기, 변경된 파일만 쓰기, 낡은 생성 파일 정리), 증분 실행
`SheetConverter.h/.cpp`	시트 → JSON 변환. 헤더 해석 (`Name:type[min..max]?`), 셀 값 검증
`RowSchema.h`	헤더에서 읽은 스키마, Enum/Const 정의 구조체, 타입 표 (`FindTypeInfo`, `FindBaseInfo`)
`DefinitionSheets.h/.cpp`	`Enum`, `Const` 시트 해석과 검증
`RowGenerator.h/.cpp`	스키마 → C++/C# 코드 생성
`ValueFormat.h/.cpp`	숫자·문자열 표기 규칙(JSON 출력용)
`Utf8Literal.h`	`U8("...")` 매크로. 한글 리터럴을 항상 UTF-8 바이트로 만듦

타입 표

`RowSchema.h`의 표 하나가 기본 타입과 enum 기반형의 모든 매핑을 가집니다. 타입을 추가하거나 바꿀 때는 이 표만 고칩니다.

이름	C++ 멤버	C++ 상수	C#	rapidjson	정수 범위
`int`	`int32`	`std::int32_t`	`int`	`GetInt()`	-2147483648 .. 2147483647
`uint`	`uint32`	`std::uint32_t`	`uint`	`GetUint()`	0 .. 4294967295
`float`	`double`	`double`	`double`	`GetDouble()`	해당 없음
`string`	`_tstring`	`const TCHAR*`	`string`	`GetString()`	해당 없음
`bool`	`bool`	`bool`	`bool`	`GetBool()`	해당 없음

enum 기반형(`Base` 열)은 `uint8`, `uint16`, `uint32`, `int32`이고 C++에서는 `std::uintN_t` / `std::int32_t`, C#에서는 `byte` / `ushort` / `uint` / `int`가 됩니다.

3. 엑셀 작성 규칙

시트와 행

- 시트 이름이 곧 `<시트>.json` 파일 이름, JSON 최상위 키, C++ 구조체 이름(`S<시트>Row`)이 됩니다. 예) `Monster` 시트 → `Monster.json` = `{"Monster":[...]}`

- 1행은 헤더, 2행부터 데이터입니다.
- 첫 칸이 `#`로 시작하는 행과 완전히 빈 행은 건너뜁니다.
- 이름이 `_`로 시작하는 시트(예: `_README`)는 변환하지 않습니다. (`--sheets`로 직접 지정하면 변환합니다.)
- 예약 시트 이름 `Enums`, `Consts`, `EnumStrings`, `GeneratedCommon`은 생성 파일 이름과 겹치므로 시트 이름으로 쓸 수 없습니다.

헤더 문법

`필드명:타입[min..max]?`

요소	설명
필드명	JSON 키이자 C++/C# 멤버 이름의 원본. 영문, 숫자, `_`만, 숫자로 시작 불가. C++/C# 키워드는 쓸 수 없고 `kFile`, `kTable`, `Specs`, `ToJSON`, `FromJSON`, `enumOk`도 예약되어 있음
타입	`int`, `uint`, `float`, `string`, `bool`, 또는 `Enum` 시트에 정의한 enum 이름 (예: `Shape:ESkillShape`)
`[min..max]`	`int`/`uint`의 허용 범위. 한쪽 생략 가능. 예) `uint[1..]`, `int[-1000..1000]`, `uint[..100]`. enum에는 쓸 수 없음
`?`	선택 열. 빈 칸이면 오류 없이 JSON 필드를 생략하고, C++은 `HasMember`로 확인해 기본값을 둠

범위를 쓰지 않아도 값은 C++ 멤버 타입(`int32`/`uint32`)에 담기는 범위여야 합니다.

셀 값 규칙

타입	규칙
`int`, `uint`	정수여야 함. 소수는 오류. `uint`는 음수 불가. 헤더 범위 또는 타입 범위를 벗어나면 오류
`float`	숫자
`string`	빈 칸은 오류 (`?`가 있으면 필드 생략)
`bool`	`TRUE` / `FALSE`, `1` / `0`, `yes` / `no`, `y` / `n`
enum	멤버 이름이 정확히 일치해야 함. 오타는 허용 목록과 함께 오류

Enum 시트

열: `Enum`, `Name`, `Value`(선택), `Base`(선택), `Comment`(선택)

- 같은 `Enum` 이름의 행들이 한 enum이 되고 순서는 시트의 행 순서입니다.
- `Value`를 비우면 앞 값 + 1이고 첫 값은 0입니다.
- `Base`는 `uint8`(기본), `uint16`, `uint32`, `int32` 중 하나이며 enum의 첫 행에만 써도 됩니다. 값은 기반형 범위 안이어야 합니다.
- 멤버 이름 중복, 값 중복, C++ 키워드 이름은 오류입니다.

Const 시트

열: `Name`, `Type`(`int`, `uint`, `float`, `string`, `bool`), `Value`, `Comment`(선택)

- Name은 식별자여야 하고 중복될 수 없습니다. 값은 Type에 맞아야 합니다.

4. 명령행

```
XlsxToJson64.exe <file.xlsx> --out <dir> [--cpp-out <dir>] [--cs-out <dir> [--cs-namespace <name>]] [--sheets Name ...] [--incremental]
```

옵션	설명
`--out`	JSON 출력 폴더 (필수)
`--cpp-out`	C++ 헤더 출력 폴더
`--cs-out`	C# 출력 폴더

옵션	설명
`--cs-namespace`	C# 네임스페이스 (기본 `GameDataRows`)
`--sheets`	일부 시트만 JSON 변환. 코드 생성과 함께 쓸 수 없음 (일부 시트만으로 만든 헤더는 다른 구조체를 지우기 때문)
`--incremental`	엑셀의 수정 시각/크기가 마지막 실행 때와 같고 출력 파일이 모두 있으면 엑셀을 열지 않고 종료

종료 코드: `0` 성공, `1` 변환 실패(데이터 오류), `2` 사용법/파일 오류

파일 쓰기 규칙

- 생성 파일은 **BOM 있는 UTF-8**, JSON은 BOM 없는 UTF-8입니다.
- 내용이 같은 파일은 다시 쓰지 않습니다(줄바꿈 차이는 무시). 불필요한 재빌드를 막습니다.
- 파일은 임시 파일에 쓴 뒤 교체하므로 읽는 쪽이 반쯤 쓰인 파일을 보지 않습니다.
- 이번에 만들지 않은 `*.gen.h` / `*.gen.cs` 중 상단에 "자동 생성 파일" 표식이 있는 것만 지웁니다. 손으로 만든 파일은 건드리지 않습니다.

증분 실행

- 상태는 `--out` 폴더의 `.XlsxToJson.stamp`에 저장합니다. 버전 관리에서 제외하세요.
- 상태에는 실행 인자, 실행 파일의 수정 시각, 엑셀의 수정 시각/크기, 만들어진 출력 파일 목록을 기록하고, 모든 출력이 끝난 뒤 마지막에 씁니다.
- 인자가 바뀌었거나 도구를 다시 빌드했거나 엑셀이 달라졌거나 출력 파일이 하나라도 없으면 다시 변환합니다.
- 엑셀을 읽을 수 없으면(Excel에 열려 있는 경우 등) 같은 인자로 만든 출력이 있을 때 **경고만 내고 종료 코드 0**으로 끝나 기존 출력으로 빌드를 계속합니다.

5. 생성되는 C++ 코드

파일	내용
`<시트>.gen.h`	`GameDataRows::S<시트>Row` 구조체
`Enums.gen.h`	`GameDataEnums`의 enum class들. 다른 헤더에 의존하지 않음
`EnumStrings.gen.h`	`ToString(enum)`, `TryParse(const TCHAR*, enum&)`
`Consts.gen.h`	`GameDataConsts::k<Name>` constexpr 상수

파일	내용
`GeneratedCommon.h`	위 파일들을 한곳에서 include하는 대표 헤더 (정의 헤더 먼저)

행 구조체

```
struct SHeroRow
{
    uint32   id = 0;
    _tstring name;
    int32    maxHp = 0;
    // ...

    static constexpr const TCHAR* kFile  = _T("Hero.json");
    static constexpr const TCHAR* kTable = _T("Hero");

    static std::initializer_list<SFieldSpec> Specs();   // 필드 이름, 종류, 범위, 선택 여부
    void ToJSON(_tValue& value, _tDocument::AllocatorType& allocator) const;
    void FromJSON(const _tValue& value);
};
```

- 멤버 이름은 필드명의 첫 글자를 소문자로 한 이름이고, 멤버 접근은 `this->`로 합니다.
- `Specs()`는 헤더의 타입과 `[min..max]`를 그대로 담아 로더가 범위를 검증하는 데 씁니다.
- 선택 열은 `Specs`에 선택 표시가 붙고 `FromJSON`에서 `HasMember`로 확인합니다.
- enum 필드가 있는 시트는 `enumOk` 멤버를 가집니다. JSON의 멤버 이름이 `TryParse`로 변환되지 않으면 `false`입니다.

로더에서 쓰는 방법

```
#include "../GameCommon/Generated/GeneratedCommon.h"   // 시트별 헤더를 하나씩 include하지 않는다
using namespace GameDataRows;
```

- 로더는 `S<시트>Row::kFile`, `Specs()`, `FromJSON()`을 사용하는 `ReadTable`로 JSON을 읽고, enum 필드가 있는 시트는 `enumOk`를 확인합니다.
- `SkillData.h`는 `using GameDataEnums::ESkillShape;`처럼 생성된 enum을 이름만 가져와 기존 코드가 그대로 동작합니다. 상수는 `GameDataConsts::k<Name>`을 참조합니다.
- 로더는 생성된 enum과 서버의 enum이 같은 타입인지 `static_assert`로 확인합니다.
- `BlockMapFile.cpp`는 `Block.gen.h`의 `SBlockRow`를 사용합니다.

6. 생성되는 C# 코드

파일	내용
`<시트>.gen.cs`	`<시트>Row` (`FileName` , `TableKey` , 속성, `Validate`) 와 `<시트>Table` (JSON 최상위 객체)
`Enums.gen.cs`	enum들
`Consts.gen.cs`	`GameConsts` 의 `public const` 상수

- 속성 이름이 JSON 키와 같아 별도 특성이 필요 없습니다.
- `Validate(where, errors)` 는 헤더의 `[min..max]` 를 벗어난 값을 `errors` 에 덧붙입니다. `<시트>Table.Validate(errors)` 는 모든 행을 검사합니다.
- JSON에는 enum이 멤버 이름 문자열로 저장되므로 역직렬화에 문자열 enum 변환기가 필요합니다. (`System.Text.Json` 의 `JsonStringEnumConverter` 또는 Newtonsoft의 `StringEnumConverter`)
- enum 이름이 `<시트>Row` / `<시트>Table` 이름과 겹치면 생성 단계에서 오류입니다.

7. 인코딩

- 생성 파일은 BOM 있는 UTF-8(줄바꿈 LF), JSON은 BOM 없는 UTF-8입니다.
- XlsxToJson 소스(`.cpp` / `.h`)는 **BOM 있는 UTF-8**로 저장합니다.
- MSVC는 `/utf-8` 이 없으면 일반 문자열 리터럴을 시스템 코드 페이지(한국어 Windows는 CP949)로 만듭니다. 그대로 파일에 쓰면 한글 주석이 깨지므로, 한글이 든 모든 리터럴은 `U8("...")` 로 감쌉니다. 컴파일러 옵션과 무관하게 항상 UTF-8 바이트가 나옵니다. 새 한글 문자열을 추가할 때도 `U8()` 를 씁니다.
- 엑셀 셀에서 읽은 문자열은 xlnt가 UTF-8로 주므로 영향이 없고, 주석도 영향이 없습니다.
- 프로젝트에 `/utf-8` 을 추가하면 일반 리터럴도 안전해집니다.

8. 빌드 자동화

Pre-Build Event

서버 빌드 시 엑셀 수정이 자동 반영되도록 ZoneServer 프로젝트에 설정합니다.

1. 솔루션 탐색기에서 ZoneServer 프로젝트를 우클릭하고 **속성**을 엽니다.
2. 위쪽 **구성**을 `모든 구성` , **플랫폼**을 `모든 플랫폼` 으로 바꿉니다.
3. **빌드 이벤트 → 빌드 전 이벤트**의 **명령줄**에 아래를 넣습니다.

```
"$(SolutionDir)Bin\x64\Release\XlsxToJson64.exe"
"$(SolutionDir)GameData\GameData.xlsx" --out "$(SolutionDir)GameData" --cpp-
out "$(SolutionDir)GameCommon\Generated" --cs-out
"$(SolutionDir)GameCommon\Generated\CSharp" --cs-namespace Game.Data --
incremental
```

경로는 실제 폴더 구조에 맞게 고칩니다. `$(SolutionDir)`은 끝에 `\`가 포함되어 있습니다.

상황	동작
엑셀 변경 없음	`up to date`로 엑셀을 열지 않고 종료, 빌드 계속
엑셀 변경됨	다시 변환. 내용이 같은 생성 파일은 다시 쓰지 않아 재빌드가 늘지 않음
데이터 오류	종료 코드 1로 빌드 실패. 오류 목록이 출력 창에 나오고 파일은 쓰지 않음
엑셀이 열려 읽을 수 없음	같은 인자로 만든 출력이 있으면 경고 후 종료 코드 0

프로젝트 종속성 (빌드 순서)

Pre-Build가 쓰는 `XlsxToJson64.exe`가 ZoneServer보다 먼저 빌드되게 합니다.

1. XlsxToJson 프로젝트가 솔루션에 없으면 솔루션을 우클릭해 **추가 → 기존 프로젝트**로 넣습니다.
2. ZoneServer 프로젝트를 우클릭하고 **빌드 종속성 → 프로젝트 종속성...**을 선택합니다.
3. **종속성** 탭에서 프로젝트 드롭다운을 `ZoneServer`로 두고 목록의 **XlsxToJson**에 체크합니다.
4. **빌드 순서** 탭에서 XlsxToJson이 ZoneServer보다 위에 있는지 확인합니다.

주의할 점은 세 가지입니다.

- exe 출력 폴더와 Pre-Build 명령줄의 경로가 같아야 합니다.
- Debug로 빌드하면 XlsxToJson도 Debug로 만들어집니다. 명령줄의 `Release`를 `$(Configuration)`으로 바꾸면 구성에 따라갑니다.
- 솔루션 **구성 관리자**에서 ZoneServer가 빌드되는 구성에 XlsxToJson의 **빌드**가 켜져 있어야 합니다.

종속성을 걸지 않는다면 XlsxToJson을 미리 한 번 빌드해 두고, 소스를 고칠 때마다 직접 다시 빌드합니다.

9. 도입 절차

1. XlsxToJson 프로젝트를 Windows에서 빌드합니다. (xInt 필요)
2. 회귀 테스트를 실행해 모두 통과하는지 확인합니다. (10절)

3. `XlsxToJson.bat`을 실행해 JSON과 생성 파일을 만들고 생성 파일의 한글 주석이 정상인지 확인합니다.
4. `GameCommon\Generated\`의 생성 헤더를 프로젝트에 **파일로 추가**합니다. (컴파일 항목이 아니라 헤더 항목)
5. 로더는 `GeneratedCommon.h` 하나만 include합니다.
6. 솔루션을 빌드합니다.
7. (선택) 8절의 Pre-Build Event와 프로젝트 종속성을 설정합니다.

10. 회귀 테스트

`XlsxToJson.Tests`는 골든 파일 비교 테스트입니다. 케이스마다 `sheets\*.tsv`로 `.xlsx`를 만들어 실제 XlsxToJson을 실행하고, 결과를 `expect\`의 정답 파일과 **바이트 단위**로 비교합니다.

```
pip install openpyxl
cd XlsxToJson.Tests
python run_tests.py --exe ..\Bin\x64\Release\XlsxToJson64.exe
```

케이스	확인 내용
`basic`	샘플 GameData.xlsx 전체 (JSON, C++, C#)
`enum_const`	enum 자동 증가값, uint32 기반 2^31 이상 값, 선택 enum 열, 모든 Const 타입, 문자열 이스케이프
`err_header`	헤더 문법 오류들
`err_enum_const`	Enum/Const 시트 오류들
`err_data`	범위, enum 오타, 정수 아님, bool 아님, 빈 칸
`err_reserved`	예약 시트 이름
`err_cpp_keyword`	C++ 키워드를 enum/멤버 이름으로 쓴 경우
`scn/*`	BOM 유무, 같은 내용은 다시 쓰지 않음, 낡은 생성 파일 정리, `--incremental` 동작

- 실패 케이스는 종료 코드 1, `expect_stderr.txt`의 문장이 stderr에 모두 있음, 출력 파일이 하나도 없음을 확인합니다.
- 생성 코드 형식을 바꾼 경우에만 `--update`로 정답 파일을 갱신하고, 바뀐 내용을 diff로 검토합니다. 오류 메시지를 바꾸면 `expect_stderr.txt`도 함께 고칩니다.

11. 문제 해결

증상	원인과 조치
생성 파일의 한글 주석이 깨짐	한글 리터럴에서 `U8()` 누락. 해당 리터럴을 감싸고 다시 빌드해 실행
C2440 `GameDataEnums::X` → `X`	서버 쪽 enum 정의가 생성 enum과 다른 타입. `SkillData.h` 처럼 `using GameDataEnums::X;` 로 가져오게 함
종료 코드 1과 오류 목록	엑셀 데이터 오류. `시트!행 열` 위치를 고침
종료 코드 2	인자 또는 파일 오류. 경로와 인자 확인
엑셀을 고쳤는데 `up to date`	인자가 같고 엑셀 수정 시각/크기가 같은 경우. `--incremental` 을 빼고 실행해 강제 변환
`--cpp-out` 과 `--sheets` 오류	함께 쓸 수 없음. 코드 생성은 모든 시트로 해야 함
테스트에서 차이가 나옴	골든 파일은 xlnt 없는 환경에서 TSV 스텁로 만든 것. xlnt의 셀 값 해석 차이(숫자 표기, 빈 셀)인지 diff로 확인